

Security Scan Report

Target: C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab

Scan Date: May 08, 2026 at 02:42 UTC

Scan ID: 34576a2c-0bde-4452-b074-644bf7e0527f

0 CRITICAL

2 HIGH

9 MEDIUM

2 LOW

Executive Summary

Security scan of 'C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab' identified 38 raw findings, reduced to 13 unique issues after deduplication. ■ 2 HIGH-severity issue(s) should be prioritized for remediation within the next 48 hours. 9 MEDIUM-priority issue(s) should be addressed in the current development sprint. 2 LOW-priority issue(s) have been logged for backlog tracking.

Scan Statistics

Metric	Value
Total Raw Findings	38
After Deduplication	13
Critical Findings	0
High Findings	2
Medium Findings	9
Low Findings	2
Scan Duration	59.8s

Detailed Findings

[HIGH] #1: Insecure Deserialization in C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:282

Risk Score	0.6076
Affected Location	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:282
Vulnerability Type	INSECURE_DESERIALIZATION
CWE ID	CWE-502: Deserialization of Untrusted Data
Detected By	semgrep

Explanation
Insecure deserialization occurs when untrusted data is deserialized without validation, potentially allowing attackers to execute arbitrary code or manipulate application logic.

Code or HTTP Excerpt

```
280 |         try:
281 |             decoded = base64.b64decode(payload)
>>> 282 |             session_object = pickle.loads(decoded)
283 |             return jsonify({"status": "restored", "object": repr(session_object)})
284 |         except Exception as exc:
```

Business Impact
An attacker could execute arbitrary code on the server or manipulate application state to bypass authentication or authorization.

Fix Recommendation
Rule-based recommendation: Validate and whitelist deserialized types. Avoid using pickle or yaml.load() with untrusted input. Use yaml.safe_load() or JSON for data interchange. Implement integrity checks on serialized data.

LLM recommendation: Validate and whitelist deserialized types. Avoid using pickle or yaml.load() with untrusted input. Use yaml.safe_load() or JSON for data interchange. Implement integrity checks on serialized data.

[MEDIUM] #2: Unknown in C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:70

Risk Score	0.5390
Affected Location	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab\app.py:70
Vulnerability Type	UNKNOWN
CWE ID	CWE-704: Incorrect Type Conversion or Cast
Detected By	semgrep

Explanation
A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
68 |         email = request.form.get("email", "")
69 |         password = request.form.get("password", "")
>>> 70 |         query = f"SELECT * FROM users WHERE email = '{email}' AND password = '{password}'"
71 |         try:
72 |             conn = connect()
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[HIGH] #3: Sql Injection in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:68

Risk Score	0.5170
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:68
Vulnerability Type	SQL_INJECTION
CWE ID	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Detected By	semgrep

Explanation

SQL Injection occurs when user-controlled input is directly concatenated into SQL queries, allowing an attacker to manipulate the query logic. This can lead to unauthorized data access, modification, or deletion of database records.

Code or HTTP Excerpt

```
66 |         error = None
67 |         if request.method == "POST":
>>> 68 |             email = request.form.get("email", "")
69 |             password = request.form.get("password", "")
70 |             query = f"SELECT * FROM users WHERE email = '{email}' AND password = '{password}'"
```

Business Impact

An attacker could read, modify, or delete all data in the database, potentially exposing customer records and sensitive business data.

Fix Recommendation

Rule-based recommendation: Use parameterized queries instead of string concatenation. In Python, use placeholder syntax: cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,)). For ORMs, use the built-in query builder instead of raw SQL.

LLM recommendation: Use parameterized queries instead of string concatenation. In Python, use placeholder syntax: cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,)). For ORMs, use the built-in query builder instead of raw SQL.

[MEDIUM] #4: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\admin.html:22

Risk Score	0.4747
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\admin.html:22
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation

Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```
20 |         <button class="btn primary" type="submit">Run</button>
21 |     </form>
>>> 22 |     <pre class="console">{{ command_output|safe }}</pre>
23 | </div>
24 | <div class="card">
```

Business Impact

An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[MEDIUM] #5: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:10

Risk Score	0.4508
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:10
Vulnerability Type	UNKNOWN
CWE ID	CWE-611: Improper Restriction of XML External Entity Reference
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
8 | import subprocess
9 | import urllib.request
```

```
>>> 10 | import xml.etree.ElementTree as ET
      11 | from pathlib import Path
      12 | from urllib.parse import unquote, urlparse
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[MEDIUM] #6: Open Redirect in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:87

Risk Score	0.4508
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:87
Vulnerability Type	OPEN_REDIRECT
CWE ID	CWE-601: URL Redirection to Untrusted Site ('Open Redirect')
Detected By	semgrep

Explanation

Open redirect vulnerabilities allow attackers to redirect users to malicious websites by manipulating URL parameters, which can be used for phishing attacks.

Code or HTTP Excerpt

```
      85 | @app.route("/logout")
      86 | def logout():
>>> 87 |     next_url = request.args.get("next", url_for("index"))
      88 |     response = redirect(next_url)
      89 |     response.delete_cookie("bank_uid")
```

Business Impact

Attackers could use your application to redirect users to phishing sites, potentially stealing credentials.

Fix Recommendation

Rule-based recommendation: Validate redirect URLs against a whitelist of allowed domains. Use relative paths instead of full URLs for internal redirects. Reject any redirect URLs pointing to external domains.

LLM recommendation: Validate redirect URLs against a whitelist of allowed domains. Use relative paths instead of full URLs for internal redirects. Reject any redirect URLs pointing to external domains.

[MEDIUM] #7: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:313

Risk Score	0.4508
------------	--------

Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:313
Vulnerability Type	UNKNOWN
CWE ID	CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
311 | cmd = request.args.get("cmd", "status")
312 | shell_cmd = f"echo Reconciled ledger for {cmd}"
>>> 313 | proc = subprocess.run(shell_cmd, shell=True, capture_output=True, text=True)
314 | summary = fetch_one(
315 |     "SELECT COUNT(*) AS users, (SELECT COUNT(*) FROM accounts) AS accounts, (SELECT COUNT(*)
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[MEDIUM] #8: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\laccount.html:24

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\laccount.html:24
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation

Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```
22 | {% for tx in transactions %}
23 |     <tr>
>>> 24 |         <td>{{ tx.counterparty|safe }}</td>
25 |         <td>{{ '%.2F'|format(tx.amount) }}</td>
26 |         <td>{{ tx.note|safe }}</td>
```

Business Impact

An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[MEDIUM] #9: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\dashboard.html:61

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\dashboard.html:61
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation
Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```
>>> 59 |         <td>{{ tx.counterparty }}</td>
      60 |         <td>{{ '%.2f'|format(tx.amount) }}</td>
      61 |         <td>{{ tx.note|safe }}</td>
      62 |     </tr>
      63 | {% endfor %}
```

Business Impact
An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[MEDIUM] #10: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\reflect.html:7

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\reflect.html:7
Vulnerability Type	XSS

CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation
Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```
5 | <h1>Customer Notice Board</h1>
6 | <p class="subtle">This page intentionally renders query params unsafely.</p>
>>> 7 | <div class="alert">Hello {{ name|safe }}, message: {{ message|safe }}</div>
8 | <form method="get" class="form inline" action="/reflect">
9 |   <label>Name<input type="text" name="name" value="{{ name|safe }}"></label>
```

Business Impact
An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation
Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[MEDIUM] #11: Xss in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\support.html:23

Risk Score	0.4042
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\templates\support.html:23
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation
Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```
21 | <td>{{ ticket.name }}</td>
22 | <td>{{ ticket.subject }}</td>
>>> 23 | <td>{{ ticket.body|safe }}</td>
24 | </tr>
25 | {% endfor %}
```

Business Impact
An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation

Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[LOW] #12: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:248

Risk Score	0.3384
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:248
Vulnerability Type	UNKNOWN
CWE ID	CWE-939: Improper Authorization in Handler for Custom URL Scheme
Detected By	semgrep

Explanation
A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
246 | def fetch_remote():
247 |     target = request.args.get("url", "http://127.0.0.1:5000/")
>>> 248 |     with urllib.request.urlopen(target, timeout=3) as handle:
249 |         body = handle.read(3000).decode("utf-8", errors="replace")
250 |     return Response(body, mimetype="text/plain")
```

Business Impact
This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[LOW] #13: Unknown in C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:369

Risk Score	0.2820
Affected Location	C:\Users\Dhruv\Desktop\CyberScurity Project\Test\vuln_bank_lab\app.py:369
Vulnerability Type	UNKNOWN
CWE ID	CWE-489: Active Debug Code
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
367 |  
368 | if __name__ == "__main__":  
>>> 369 |     app.run(host="127.0.0.1", port=5001, debug=True)
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

Appendix

Deduplication Statistics

Total raw findings: 38 | After deduplication: 13 | Reduction: 66%

Scan Metadata

Scan ID	34576a2c-0bde-4452-b074-644bf7e0527f
Target	C:\Users\Dhruv\Desktop\CyberSceurity Project\Test\vuln_bank_lab
Timestamp	2026-05-08T02:42:23.306558+00:00
Duration	59.8s
Scanners Used	semgrep
Scanner Versions	semgrep=1.159.0

Raw Findings Data (JSON)

```
[
  {
    "id": "2e8ad5a0-5deb-4687-bd1c-7e413387a35b",
    "title": "Insecure Deserialization in C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 282,
    "line_end": 282,
    "severity": "HIGH",
    "vuln_type": "INSECURE_DESERIALIZATION",
    "description": "Detected the use of an insecure deserialization library in a Flask route. These libraries are vulnerable to deserialization attacks.",
    "rule_id": "python.flask.security.insecure-deserialization.insecure-deserialization",
    "confidence": 0.9,
    "cwe_id": "CWE-502: Deserialization of Untrusted Data",
    "cvss_score": null,
    "code_snippet": "
      280 |         try:
      281 |             decoded = base64.b64decode(payload)
      282 |             return jsonify(decoded)
    ",
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.6076,
    "risk_category": "HIGH",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Insecure deserialization occurs when untrusted data is deserialized without validation. This can lead to arbitrary code execution or data manipulation.",
    "fix_suggestion": "Validate and whitelist deserialized types. Avoid using pickle or yaml.load() with untrusted data.",
    "business_impact": "An attacker could execute arbitrary code on the server or manipulate application state.",
    "raw": {
      "check_id": "python.flask.security.insecure-deserialization.insecure-deserialization",
      "path": "C:\\Users\\Dhruv\\Desktop\\CyberSceurity Project\\Test\\vuln_bank_lab\\app.py",
      "start": {
        "line": 282,
        "col": 26,
        "offset": 9736
      },
      "end": {
        "line": 282,
        "col": 47,
        "offset": 9757
      },
      "extra": {

```

```

    "message": "Detected the use of an insecure deserialization library in a Flask route. These libraries are insecure and can be exploited to compromise application security and potentially leak sensitive data.",
    "metadata": {
      "owasp": [
        "A08:2017 - Insecure Deserialization",
        "A08:2021 - Software and Data Integrity Failures",
        "A08:2025 - Software or Data Integrity Failures"
      ],
      "cwe": [
        "CWE-502: Deserialization of Untrusted Data"
      ],
      "references": [
        "https://docs.python.org/3/library/pickle.html"
      ],
      "category": "security",
      "technology": [
        "flask"
      ],
      "cwe2022-top25": true,
      "cwe2021-top25": true,
      "subcategory": [
        "audit"
      ],
      "likelihood": "LOW",
      "impact": "MEDIUM",
      "confidence": "LOW",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "Insecure Deserialization "
      ],
      "source": "https://semgrep.dev/r/python.flask.security.insecure-deserialization.insecure-deserialization",
      "shortlink": "https://sg.run/N45z"
    },
    "severity": "ERROR",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
}
},
{
  "id": "48b92298-47f2-4e62-baef-93695f9e7e5c",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:70",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 70,
  "line_end": 70,
  "severity": "HIGH",
  "vuln_type": "UNKNOWN",
  "description": "Detected user input used to manually construct a SQL string. This is usually bad practice and can be exploited to compromise application security and potentially leak sensitive data.",
  "rule_id": "python.flask.security.injection.tainted-sql-string.tainted-sql-string",
  "confidence": 0.9,
  "cwe_id": "CWE-704: Incorrect Type Conversion or Cast",
  "cvss_score": null,
  "code_snippet": "
        68 |         email = request.form.get('email', '')\n
        69 |         password = request.form.get('password', '')\n
        70 |         # Validate email and password\n
        71 |         if not email or not password:\n
        72 |             return jsonify({'error': 'Email and password are required'}), 400\n
        73 |         # Hash password\n
        74 |         hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())\n
        75 |         # Check if user exists\n
        76 |         user = db.session.query(User).filter(User.email == email).first()\n
        77 |         if user:\n
        78 |             # Verify password\n
        79 |             if not bcrypt.checkpw(password.encode('utf-8'), user.password):\n
        80 |                 return jsonify({'error': 'Invalid password'}), 401\n
        81 |             # User exists and password is correct\n
        82 |             return jsonify({'message': 'Login successful', 'user': user.to_dict()}), 200\n
        83 |         else:\n
        84 |             # User does not exist\n
        85 |             return jsonify({'error': 'User not found'}), 404\n
        86 |     \n
        87 |     @app.route('/register', methods=['POST'])\n
        88 |     def register():\n
        89 |         email = request.form.get('email', '')\n
        90 |         password = request.form.get('password', '')\n
        91 |         # Validate email and password\n
        92 |         if not email or not password:\n
        93 |             return jsonify({'error': 'Email and password are required'}), 400\n
        94 |         # Check if user exists\n
        95 |         user = db.session.query(User).filter(User.email == email).first()\n
        96 |         if user:\n
        97 |             return jsonify({'error': 'Email already exists'}), 400\n
        98 |         # Hash password\n
        99 |         hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())\n
        100 |         # Create new user\n
        101 |         new_user = User(email=email, password=hashed_password)\n
        102 |         db.session.add(new_user)\n
        103 |         db.session.commit()\n
        104 |         return jsonify({'message': 'Registration successful', 'user': new_user.to_dict()}), 201\n
        105 |     \n
        106 |     @app.route('/logout', methods=['POST'])\n
        107 |     def logout():\n
        108 |         return jsonify({'message': 'Logout successful'}), 200\n
        109 |     \n
        110 |     @app.route('/forgot_password', methods=['POST'])\n
        111 |     def forgot_password():\n
        112 |         email = request.form.get('email', '')\n
        113 |         # Check if user exists\n
        114 |         user = db.session.query(User).filter(User.email == email).first()\n
        115 |         if user:\n
        116 |             # Generate reset token\n
        117 |             token = generate_reset_token(email)\n
        118 |             # Send email with reset link\n
        119 |             send_email(email, token)\n
        120 |             return jsonify({'message': 'Reset link sent to your email'}), 200\n
        121 |         else:\n
        122 |             return jsonify({'error': 'Email not found'}), 404\n
        123 |     \n
        124 |     @app.route('/reset_password', methods=['POST'])\n
        125 |     def reset_password():\n
        126 |         token = request.form.get('token', '')\n
        127 |         new_password = request.form.get('password', '')\n
        128 |         # Verify token\n
        129 |         email = verify_reset_token(token)\n
        130 |         if email:\n
        131 |             # Update password\n
        132 |             user = db.session.query(User).filter(User.email == email).first()\n
        133 |             user.password = bcrypt.hashpw(new_password.encode('utf-8'), bcrypt.gensalt())\n
        134 |             db.session.commit()\n
        135 |             return jsonify({'message': 'Password reset successful'}), 200\n
        136 |         else:\n
        137 |             return jsonify({'error': 'Invalid token'}), 400\n
        138 |     \n
        139 |     @app.route('/change_password', methods=['POST'])\n
        140 |     def change_password():\n
        141 |         current_password = request.form.get('current_password', '')\n
        142 |         new_password = request.form.get('password', '')\n
        143 |         # Verify current password\n
        144 |         user = db.session.query(User).filter(User.email == email).first()\n
        145 |         if user and bcrypt.checkpw(current_password.encode('utf-8'), user.password):\n
        146 |             # Update password\n
        147 |             user.password = bcrypt.hashpw(new_password.encode('utf-8'), bcrypt.gensalt())\n
        148 |             db.session.commit()\n
        149 |             return jsonify({'message': 'Password changed successfully'}), 200\n
        150 |         else:\n
        151 |             return jsonify({'error': 'Current password is incorrect'}), 400\n
        152 |     \n
        153 |     @app.route('/profile', methods=['GET'])\n
        154 |     def profile():\n
        155 |         user = db.session.query(User).filter(User.email == email).first()\n
        156 |         if user:\n
        157 |             return jsonify({'user': user.to_dict()}), 200\n
        158 |         else:\n
        159 |             return jsonify({'error': 'User not found'}), 404\n
        160 |     \n
        161 |     @app.route('/update_profile', methods=['POST'])\n
        162 |     def update_profile():\n
        163 |         new_email = request.form.get('email', '')\n
        164 |         new_password = request.form.get('password', '')\n
        165 |         # Verify current password\n
        166 |         user = db.session.query(User).filter(User.email == email).first()\n
        167 |         if user and bcrypt.checkpw(current_password.encode('utf-8'), user.password):\n
        168 |             # Update email and password\n
        169 |             user.email = new_email\n
        170 |             user.password = bcrypt.hashpw(new_password.encode('utf-8'), bcrypt.gensalt())\n
        171 |             db.session.commit()\n
        172 |             return jsonify({'message': 'Profile updated successfully'}), 200\n
        173 |         else:\n
        174 |             return jsonify({'error': 'Current password is incorrect'}), 400\n
        175 |     \n
        176 |     @app.route('/delete_account', methods=['POST'])\n
        177 |     def delete_account():\n
        178 |         # Delete user\n
        179 |         user = db.session.query(User).filter(User.email == email).first()\n
        180 |         if user:\n
        181 |             db.session.delete(user)\n
        182 |             db.session.commit()\n
        183 |             return jsonify({'message': 'Account deleted successfully'}), 200\n
        184 |         else:\n
        185 |             return jsonify({'error': 'User not found'}), 404\n
        186 |     \n
        187 |     @app.route('/health', methods=['GET'])\n
        188 |     def health():\n
        189 |         return jsonify({'status': 'ok'}), 200\n
        190 |     \n
        191 |     @app.route('/api/v1/health', methods=['GET'])\n
        192 |     def api_health():\n
        193 |         return jsonify({'status': 'ok'}), 200\n
        194 |     \n
        195 |     @app.route('/api/v1/status', methods=['GET'])\n
        196 |     def api_status():\n
        197 |         return jsonify({'status': 'ok'}), 200\n
        198 |     \n
        199 |     @app.route('/api/v1/info', methods=['GET'])\n
        200 |     def api_info():\n
        201 |         return jsonify({'version': '1.0.0', 'author': 'Dhruv', 'email': 'dhruv@example.com'}), 200\n
        202 |     \n
        203 |     @app.route('/api/v1/docs', methods=['GET'])\n
        204 |     def api_docs():\n
        205 |         return jsonify({'docs': 'https://api.example.com/docs'}), 200\n
        206 |     \n
        207 |     @app.route('/api/v1/privacy', methods=['GET'])\n
        208 |     def api_privacy():\n
        209 |         return jsonify({'privacy': 'https://api.example.com/privacy'}), 200\n
        210 |     \n
        211 |     @app.route('/api/v1/terms', methods=['GET'])\n
        212 |     def api_terms():\n
        213 |         return jsonify({'terms': 'https://api.example.com/terms'}), 200\n
        214 |     \n
        215 |     @app.route('/api/v1/contact', methods=['GET'])\n
        216 |     def api_contact():\n
        217 |         return jsonify({'contact': 'https://api.example.com/contact'}), 200\n
        218 |     \n
        219 |     @app.route('/api/v1/about', methods=['GET'])\n
        220 |     def api_about():\n
        221 |         return jsonify({'about': 'https://api.example.com/about'}), 200\n
        222 |     \n
        223 |     @app.route('/api/v1/help', methods=['GET'])\n
        224 |     def api_help():\n
        225 |         return jsonify({'help': 'https://api.example.com/help'}), 200\n
        226 |     \n
        227 |     @app.route('/api/v1/faq', methods=['GET'])\n
        228 |     def api_faq():\n
        229 |         return jsonify({'faq': 'https://api.example.com/faq'}), 200\n
        230 |     \n
        231 |     @app.route('/api/v1/feedback', methods=['POST'])\n
        232 |     def api_feedback():\n
        233 |         feedback = request.form.get('feedback', '')\n
        234 |         # Save feedback\n
        235 |         db.session.add(Feedback(email=email, feedback=feedback))\n
        236 |         db.session.commit()\n
        237 |         return jsonify({'message': 'Feedback received'}), 200\n
        238 |     \n
        239 |     @app.route('/api/v1/subscribe', methods=['POST'])\n
        240 |     def api_subscribe():\n
        241 |         email = request.form.get('email', '')\n
        242 |         # Check if user is already subscribed\n
        243 |         user = db.session.query(User).filter(User.email == email).first()\n
        244 |         if user and not user.is_subscribed:\n
        245 |             # Subscribe user\n
        246 |             user.is_subscribed = True\n
        247 |             db.session.commit()\n
        248 |             return jsonify({'message': 'Subscribed successfully'}), 200\n
        249 |         else:\n
        250 |             return jsonify({'error': 'Already subscribed or email not found'}), 400\n
        251 |     \n
        252 |     @app.route('/api/v1/unsubscribe', methods=['POST'])\n
        253 |     def api_unsubscribe():\n
        254 |         email = request.form.get('email', '')\n
        255 |         # Check if user is already unsubscribed\n
        256 |         user = db.session.query(User).filter(User.email == email).first()\n
        257 |         if user and user.is_subscribed:\n
        258 |             # Unsubscribe user\n
        259 |             user.is_subscribed = False\n
        260 |             db.session.commit()\n
        261 |             return jsonify({'message': 'Unsubscribed successfully'}), 200\n
        262 |         else:\n
        263 |             return jsonify({'error': 'Already unsubscribed or email not found'}), 400\n
        264 |     \n
        265 |     @app.route('/api/v1/reset_email', methods=['POST'])\n
        266 |     def api_reset_email():\n
        267 |         email = request.form.get('email', '')\n
        268 |         # Generate new email\n
        269 |         new_email = generate_new_email(email)\n
        270 |         # Send email with new email\n
        271 |         send_email(email, new_email)\n
        272 |         return jsonify({'message': 'New email generated and sent to your email'}), 200\n
        273 |         \n
        274 |         @app.route('/api/v1/verify_email', methods=['POST'])\n
        275 |         def api_verify_email():\n
        276 |             token = request.form.get('token', '')\n
        277 |             # Verify token\n
        278 |             email = verify_email_token(token)\n
        279 |             if email:\n
        280 |                 # Verify email\n
        281 |                 user = db.session.query(User).filter(User.email == email).first()\n
        282 |                 if user:\n
        283 |                     return jsonify({'message': 'Email verified successfully'}), 200\n
        284 |                 else:\n
        285 |                     return jsonify({'error': 'Email not found'}), 404\n
        286 |             else:\n
        287 |                 return jsonify({'error': 'Invalid token'}), 400\n
        288 |         \n
        289 |         @app.route('/api/v1/confirm_email', methods=['POST'])\n
        290 |         def api_confirm_email():\n
        291 |             token = request.form.get('token', '')\n
        292 |             # Verify token\n
        293 |             email = verify_email_token(token)\n
        294 |             if email:\n
        295 |                 # Confirm email\n
        296 |                 user = db.session.query(User).filter(User.email == email).first()\n
        297 |                 if user:\n
        298 |                     return jsonify({'message': 'Email confirmed successfully'}), 200\n
        299 |                 else:\n
        300 |                     return jsonify({'error': 'Email not found'}), 404\n
        301 |             else:\n
        302 |                 return jsonify({'error': 'Invalid token'}), 400\n
        303 |         \n
        304 |         @app.route('/api/v1/delete_email', methods=['POST'])\n
        305 |         def api_delete_email():\n
        306 |             email = request.form.get('email', '')\n
        307 |             # Delete email\n
        308 |             user = db.session.query(User).filter(User.email == email).first()\n
        309 |             if user:\n
        310 |                 db.session.delete(user)\n
        311 |                 db.session.commit()\n
        312 |                 return jsonify({'message': 'Email deleted successfully'}), 200\n
        313 |             else:\n
        314 |                 return jsonify({'error': 'Email not found'}), 404\n
        315 |         \n
        316 |         @app.route('/api/v1/revoke_email', methods=['POST'])\n
        317 |         def api_revoke_email():\n
        318 |             token = request.form.get('token', '')\n
        319 |             # Revoke token\n
        320 |             email = revoke_email_token(token)\n
        321 |             if email:\n
        322 |                 return jsonify({'message': 'Token revoked successfully'}), 200\n
        323 |             else:\n
        324 |                 return jsonify({'error': 'Invalid token'}), 400\n
        325 |         \n
        326 |         @app.route('/api/v1/refresh_email', methods=['POST'])\n
        327 |         def api_refresh_email():\n
        328 |             token = request.form.get('token', '')\n
        329 |             # Refresh token\n
        330 |             email = refresh_email_token(token)\n
        331 |             if email:\n
        332 |                 return jsonify({'message': 'Token refreshed successfully'}), 200\n
        333 |             else:\n
        334 |                 return jsonify({'error': 'Invalid token'}), 400\n
        335 |         \n
        336 |         @app.route('/api/v1/logout_email', methods=['POST'])\n
        337 |         def api_logout_email():\n
        338 |             token = request.form.get('token', '')\n
        339 |             # Logout token\n
        340 |             email = logout_email_token(token)\n
        341 |             if email:\n
        342 |                 return jsonify({'message': 'Token logged out successfully'}), 200\n
        343 |             else:\n
        344 |                 return jsonify({'error': 'Invalid token'}), 400\n
        345 |         \n
        346 |         @app.route('/api/v1/login_email', methods=['POST'])\n
        347 |         def api_login_email():\n
        348 |             email = request.form.get('email', '')\n
        349 |             password = request.form.get('password', '')\n
        350 |             # Login user\n
        351 |             user = db.session.query(User).filter(User.email == email).first()\n
        352 |             if user and bcrypt.checkpw(password.encode('utf-8'), user.password):\n
        353 |                 return jsonify({'message': 'Login successful', 'user': user.to_dict()}), 200\n
        354 |             else:\n
        355 |                 return jsonify({'error': 'Invalid email or password'}), 401\n
        356 |         \n
        357 |         @app.route('/api/v1/register_email', methods=['POST'])\n
        358 |         def api_register_email():\n
        359 |             email = request.form.get('email', '')\n
        360 |             password = request.form.get('password', '')\n
        361 |             # Register user\n
        362 |             user = db.session.query(User).filter(User.email == email).first()\n
        363 |             if user:\n
        364 |                 return jsonify({'error': 'Email already exists'}), 400\n
        365 |             else:\n
        366 |                 new_user = User(email=email, password=bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()))\n
        367 |                 db.session.add(new_user)\n
        368 |                 db.session.commit()\n
        369 |                 return jsonify({'message': 'Registration successful', 'user': new_user.to_dict()}), 201\n
        370 |         \n
        371 |         @app.route('/api/v1/forgot_email', methods=['POST'])\n
        372 |         def api_forgot_email():\n
        373 |             email = request.form.get('email', '')\n
        374 |             # Forgot password\n
        375 |             user = db.session.query(User).filter(User.email == email).first()\n
        376 |             if user:\n
        377 |                 return jsonify({'message': 'Forgot password successful', 'user': user.to_dict()}), 200\n
        378 |             else:\n
        379 |                 return jsonify({'error': 'Email not found'}), 404\n
        380 |         \n
        381 |         @app.route('/api/v1/reset_email', methods=['POST'])\n
        382 |         def api_reset_email():\n
        383 |             email = request.form.get('email', '')\n
        384 |             # Reset password\n
        385 |             user = db.session.query(User).filter(User.email == email).first()\n
        386 |             if user:\n
        387 |                 return jsonify({'message': 'Reset password successful', 'user': user.to_dict()}), 200\n
        388 |             else:\n
        389 |                 return jsonify({'error': 'Email not found'}), 404\n
        390 |         \n
        391 |         @app.route('/api/v1/change_email', methods=['POST'])\n
        392 |         def api_change_email():\n
        393 |             email = request.form.get('email', '')\n
        394 |             # Change email\n
        395 |             user = db.session.query(User).filter(User.email == email).first()\n
        396 |             if user:\n
        397 |                 return jsonify({'message': 'Change email successful', 'user': user.to_dict()}), 200\n
        398 |             else:\n
        399 |                 return jsonify({'error': 'Email not found'}), 404\n
        400 |         \n
        401 |         @app.route('/api/v1/delete_email', methods=['POST'])\n
        402 |         def api_delete_email():\n
        403 |             email = request.form.get('email', '')\n
        404 |             # Delete email\n
        405 |             user = db.session.query(User).filter(User.email == email).first()\n
        406 |             if user:\n
        407 |                 db.session.delete(user)\n
        408 |                 db.session.commit()\n
        409 |                 return jsonify({'message': 'Delete email successful', 'user': user.to_dict()}), 200\n
        410 |             else:\n
        411 |                 return jsonify({'error': 'Email not found'}), 404\n
        412 |         \n
        413 |         @app.route('/api/v1/revoke_email', methods=['POST'])\n
        414 |         def api_revoke_email():\n
        415 |             token = request.form.get('token', '')\n
        416 |             # Revoke token\n
        417 |             email = revoke_email_token(token)\n
        418 |             if email:\n
        419 |                 return jsonify({'message': 'Revoke email successful', 'user': user.to_dict()}), 200\n
        420 |             else:\n
        421 |                 return jsonify({'error': 'Invalid token'}), 400\n
        422 |         \n
        423 |         @app.route('/api/v1/refresh_email', methods=['POST'])\n
        424 |         def api_refresh_email():\n
        425 |             token = request.form.get('token', '')\n
        426 |             # Refresh token\n
        427 |             email = refresh_email_token(token)\n
        428 |             if email:\n
        429 |                 return jsonify({'message': 'Refresh email successful', 'user': user.to_dict()}), 200\n
        430 |             else:\n
        431 |                 return jsonify({'error': 'Invalid token'}), 400\n
        432 |         \n
        433 |         @app.route('/api/v1/logout_email', methods=['POST'])\n
        434 |         def api_logout_email():\n
        435 |             token = request.form.get('token', '')\n
        436 |             # Logout token\n
        437 |             email = logout_email_token(token)\n
        438 |             if email:\n
        439 |                 return jsonify({'message': 'Logout email successful', 'user': user.to_dict()}), 200\n
        440 |             else:\n
        441 |                 return jsonify({'error': 'Invalid token'}), 400\n
        442 |         \n
        443 |         @app.route('/api/v1/login_email', methods=['POST'])\n
        444 |         def api_login_email():\n
        445 |             email = request.form.get('email', '')\n
        446 |             password = request.form.get('password', '')\n
        447 |             # Login user\n
        448 |             user = db.session.query(User).filter(User.email == email).first()\n
        449 |             if user and bcrypt.checkpw(password.encode('utf-8'), user.password):\n
        450 |                 return jsonify({'message': 'Login email successful', 'user': user.to_dict()}), 200\n
        451 |             else:\n
        452 |                 return jsonify({'error': 'Invalid email or password'}), 401\n
        453 |         \n
        454 |         @app.route('/api/v1/register_email', methods=['POST'])\n
        455 |         def api_register_email():\n
        456 |             email = request.form.get('email', '')\n
        457 |             password = request.form.get('password', '')\n
        458 |             # Register user\n
        459 |             user = db.session.query(User).filter(User.email == email).first()\n
        460 |             if user:\n
        461 |                 return jsonify({'error': 'Email already exists'}), 400\n
        462 |             else:\n
        463 |                 new_user = User(email=email, password=bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()))\n
        464 |                 db.session.add(new_user)\n
        465 |                 db.session.commit()\n
        466 |                 return jsonify({'message': 'Register email successful', 'user': new_user.to_dict()}), 201\n
        467 |         \n
        468 |         @app.route('/api/v1/forgot_email', methods=['POST'])\n
        469 |         def api_forgot_email():\n
        470 |             email = request.form.get('email', '')\n
        471 |             # Forgot password\n
        472 |             user = db.session.query(User).filter(User.email == email).first()\n
        473 |             if user:\n
        474 |                 return jsonify({'message': 'Forgot email successful', 'user': user.to_dict()}), 200\n
        475 |             else:\n
        476 |                 return jsonify({'error': 'Email not found'}), 404\n
        477 |         \n
        478 |         @app.route('/api/v1/reset_email', methods=['POST'])\n
        479 |         def api_reset_email():\n
        480 |             email = request.form.get('email', '')\n
        481 |             # Reset password\n
        482 |             user = db.session.query(User).filter(User.email == email).first()\n
        483 |             if user:\n
        484 |                 return jsonify({'message': 'Reset email successful', 'user': user.to_dict()}), 200\n
        485 |             else:\n
        486 |                 return jsonify({'error': 'Email not found'}), 404\n
        487 |         \n
        488 |         @app.route('/api/v1/change_email', methods=['POST'])\n
        489 |         def api_change_email():\n
        490 |             email = request.form.get('email', '')\n
        491 |             # Change email\n
        492 |             user = db.session.query(User).filter(User.email == email).first()\n
        493 |             if user:\n
        494 |                 return jsonify({'message': 'Change email successful', 'user': user.to_dict()}), 200\n
        495 |             else:\n
        496 |                 return jsonify({'error': 'Email not found'}), 404\n
        497 |         \n
        498 |         @app.route('/api/v1/delete_email', methods=['POST'])\n
        499 |         def api_delete_email():\n
        500 |             email = request.form.get('email', '')\n
        501 |             # Delete email\n
        502 |             user = db.session.query(User).filter(User.email == email).first()\n
        503 |             if user:\n
        504 |                 db.session.delete(user)\n
        505 |                 db.session.commit()\n
        506 |                 return jsonify({'message': 'Delete email successful', 'user': user.to_dict()}), 200\n
        507 |             else:\n
        508 |                 return jsonify({'error': 'Email not found'}), 404\n
        509 |         \n
        510 |         @app.route('/api/v1/revoke_email', methods=['POST'])\n
        511 |         def api_revoke_email():\n
        512 |             token = request.form.get('token', '')\n
        513 |             # Revoke token\n
        514 |             email = revoke_email_token(token)\n
        515 |             if email:\n
        516 |                 return jsonify({'message': 'Revoke email successful', 'user': user.to_dict()}), 200\n
        517 |             else:\n
        518 |                 return jsonify({'error': 'Invalid token'}), 400\n
        519 |         \n
        520 |         @app.route('/api/v1/refresh_email', methods=['POST'])\n
        521 |         def api_refresh_email():\n
        522 |             token = request.form.get('token', '')\n
        523 |             # Refresh token\n
        524 |             email = refresh_email_token(token)\n
        525 |             if email:\n
        526 |                 return jsonify({'message': 'Refresh email successful', 'user': user.to_dict()}), 200\n
        527 |             else:\n
        528 |                 return jsonify({'error': 'Invalid token'}), 400\n
        529 |         \n
        530 |         @app.route('/api/v1/logout_email', methods=['POST'])\n
        531 |         def api_logout_email():\n
        532 |             token = request.form.get('token', '')\n
        533 |             # Logout token\n
        534 |             email = logout_email_token(token)\n
        535 |             if email:\n
        536 |                 return jsonify({'message': 'Logout email successful', 'user': user.to_dict()}), 200\n
        537 |             else:\n
        538 |                 return jsonify({'error': 'Invalid token'}), 400\n
        539 |         \n
        540 |         @app.route('/api/v1/login_email', methods=['POST'])\n
        541 |         def api_login_email():\n
        542 |             email = request.form.get('email', '')\n
        543 |             password = request.form.get('password', '')\n
        544 |             # Login user\n
        545 |             user = db.session.query(User).filter(User.email == email).first()\n
        546 |             if user and bcrypt.checkpw(password.encode('utf-8'), user.password):\n
        547 |                 return jsonify({'message': 'Login email successful', 'user': user.to_dict()}), 200\n
        548 |             else:\n
        549 |                 return jsonify({'error': 'Invalid email or password'}), 401\n
        550 |         \n
        551 |         @app.route('/api/v1/register_email', methods=['POST'])\n
        552 |         def api_register_email():\n
        553 |             email = request.form.get('email', '')\n
        554 |             password = request.form.get('password', '')\n
        555 |             # Register user\n
        556 |             user = db.session.query(User).filter(User.email == email).first()\n
        557 |             if user:\n
        558 |                 return jsonify({'error': 'Email already exists'}), 400\n
        559 |             else:\n
        560 |                 new_user = User(email=email, password=bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()))\n
        561 |                 db.session.add(new_user)\n
        562 |                 db.session.commit()\n
        563 |                 return jsonify({'message': 'Register email successful', 'user': new_user.to_dict()}), 201\n
        564 |         \n
        565 |         @app.route('/api/v1/forgot_email', methods=['POST'])\n
        566 |         def api_forgot_email():\n
        567 |             email = request.form.get('email', '')\n
        568 |             # Forgot password\n
        569 |             user = db.session.query(User).filter(User.email == email).first()\n
        570 |             if user:\n
        571 |                 return jsonify({'message': 'Forgot email successful', 'user': user.to_dict()}), 200\n
        572 |             else:\n
        573 |                 return jsonify({'error': 'Email not found'}), 404\n
        574 |         \n
        575 |         @app.route('/api/v1/reset_email', methods=['POST'])\n
        576 |         def api_reset_email():\n
        577 |             email = request.form.get('email', '')\n
        578 |             # Reset password\n
        579 |             user = db.session.query(User).filter(User.email == email).first()\n
        580 |             if user:\n
        581 |                 return jsonify({'message': 'Reset email successful', 'user': user.to_dict()}), 200\n
        582 |             else:\n
        583 |                 return jsonify({'error': 'Email not found'}), 404\n
        584 |         \n
        585 |         @app.route('/api/v1/change_email', methods=['POST'])\n
        586 |         def api_change_email():\n
        587 |             email = request.form.get('email', '')\n
        588 |             # Change email\n
        589 |             user = db.session.query(User).filter(User.email == email).first()\n
        590 |             if user:\n
        591 |                 return jsonify({'message': 'Change email successful', 'user': user.to_dict()}), 200\n
        592 |             else:\n
        593 |                 return jsonify({'error': 'Email not found'}), 404\n
        594 |         \n
        595 |         @app.route('/api/v1/delete_email', methods=['POST'])\n
        596 |         def api_delete_email():\n
        597 |             email = request.form.get('email', '')\n
        598 |             # Delete email\n
        599 |             user = db.session.query(User).filter(User.email == email).first()\n
        600 |             if user:\n
        601 |                 db.session.delete(user)\n
        602 |                 db.session.commit()\n
        603 |                 return jsonify({'message': 'Delete email successful', 'user': user.to_dict()}), 200\n
        604 |             else:\n
        605 |                 return jsonify({'error': 'Email not found'}), 404\n
        606 |         \n
        607 |         @app.route('/api/v1/revoke_email', methods=['POST'])\n
        608 |         def api_revoke_email():\n
        609 |             token = request.form.get('token', '')\n
        610 |             # Revoke token\n
        611 |             email = revoke_email_token(token)\n
        612 |             if email:\n
        613 |                 return jsonify({'message': 'Revoke email successful', 'user': user.to_dict()}), 200\n
        614 |             else:\n
        615 |                 return jsonify({'error': 'Invalid token'}), 400\n
        616 |         \n
        617 |         @app.route('/api/v1/refresh_email', methods=['POST'])\n
        618 |         def api_refresh_email():\n
        619 |             token = request.form.get('token', '')\n
        620 |             # Refresh token\n
        621 |             email = refresh_email_token(token)\n
        622 |             if email:\n
        623 |                 return jsonify({'message': 'Refresh email successful', 'user': user.to_dict()}), 200\n
        624 |             else:\n
        625 |                 return jsonify({'error': 'Invalid token'}), 400\n
        626 |         \n
        627 |         @app.route('/api/v1/logout_email', methods=['POST'])\n
        628 |         def api_logout_email():\n
        629 |             token = request.form.get('token', '')\n
        630 |             # Logout token\n
        631 |             email = logout_email_token(token)\n
        632 |             if email:\n
        633 |                 return jsonify({'message': 'Logout email successful', 'user': user.to_dict()}), 200\n
        634 |             else:\n
        635 |                 return jsonify({'error': 'Invalid token'}), 400\n
        636 |         \n
        637 |         @app.route('/api/v1/login_email', methods=['POST'])\n
        638 |         def api_login_email():\n
        639 |             email = request.form.get('email', '')\n
        640 |             password = request.form.get('password', '')\n
        641 |             # Login user\n
        642 |             user = db.session.query(User).filter(User.email == email).first()\n
        643 |             if user and bcrypt.checkpw(password.encode('utf-8'), user.password):\n
        644 |                 return jsonify({'message': 'Login email successful', 'user': user.to_dict()}), 200\n
        645 |             else:\n
        646 |                 return jsonify({'error': 'Invalid email or password'}), 401\n
        647 |         \n
        648 |         @app.route('/api/v1/register_email', methods=['POST'])\n
        649 |         def api_register_email():\n
        650 |             email = request.form.get('email', '')\n
        651 |             password = request.form.get('password', '')\n
        652 |             # Register user\n
        653 |             user = db.session.query(User).filter(User.email == email).first()\n
        654 |             if user:\n
        655 |                 return jsonify({'error': 'Email already exists'}), 400\n
        656 |             else:\n
        657 |                 new_user = User(email=email, password=bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt()))\n
        658 |                 db.session.add(new_user)\n
        659 |                 db.session.commit()\n
        660 |                 return jsonify({'message': 'Register email successful', 'user': new_user.to_dict()}), 201\n
        661 |         \n
        662 |         @app.route('/api/v1/forgot_email', methods=['POST'])\n
        663 |         def api_forgot_email():\n
        664 |             email = request.form.get('email', '')\n
        665 |             # Forgot password\n
        666 |             user = db.session.query(User).filter(User.email == email).first()\n
        667 |             if user:\n
        668 |                 return jsonify({'message': 'Forgot email successful', 'user': user.to_dict()}), 200\n
        669 |             else:\n
        670 |                 return jsonify({'error': 'Email not found'}), 404\n
        671 |         \n
        672 |         @app.route('/api/v1/reset_email', methods=['POST'])\n
        673 |         def api_reset_email():\n
        674 |             email = request.form.get('email', '')\n
        675 |             # Reset password\n
        676 |             user = db.session.query(User).filter(User.email == email).first()\n
        677 |             if user:\n
        678 |                 return jsonify({'message': 'Reset email successful', 'user': user.to_dict()}), 200\n
        679 |             else:\n
        680 |                 return jsonify({'error': 'Email not found'}), 404\n
        681 |         \n
        682 |         @app.route('/api/v1/change_email', methods=['POST'])\n
        683 |         def api_change_email():\n
        684 |             email = request.form.get('email', '')\n
        685 |             # Change email\n
        686 |             user = db.session.query(User).filter(User.email == email).first()\n
        687 |             if user:\n
        688 |                 return jsonify({'message': 'Change email successful', 'user': user.to_dict()}), 200\n
        689 |             else:\n
        690 |                 return jsonify({'error': 'Email not found'}), 404\n
        691 |         \n
        692 |         @app.route('/api/v1/delete_email', methods=['POST'])\n
        693 |         def api_delete_email():\n
        694 |             email = request.form.get('email', '')\n
        695 |             # Delete email\n
        696 |             user = db.session.query(User).filter(User.email == email).first()\n
        697 |             if user:\n
        698 |                 db.session.delete(user)\n
        699 |                 db.session.commit()\n
        700 |                 return jsonify({'message': 'Delete email successful', 'user': user.to_dict()}), 200\n
        701 |             else:\n
        702 |                 return jsonify({'error': 'Email not found'}), 404\n
        703 |         \n
        704 |         @app.route('/api/v1/revoke_email', methods=['POST'])\n
        705 |         def api_revoke_email():\n
        706 |             token = request.form.get('token', '')\n
        707 |             # Revoke token\n
        708 |             email = revoke_email_token(token)\n
        709 |             if email:\n
        710 |                 return jsonify({'message': 'Revoke email successful', 'user': user.to_dict()}), 200\n
        711 |             else:\n
        712 |                 return jsonify({'error': 'Invalid token'}), 400\n
        713 |         \n
        714 |         @app.route('/api/v1/refresh_email', methods=['POST'])\n
        715 |         def api_refresh_email():\n
        716 |             token = request.form.get('token', '')\n
        717 |             # Refresh token\n
        718 |             email = refresh_email_token(token)\n
        719 |             if email:\n
        720 |                 return jsonify({'message': 'Refresh email successful', 'user': user.to_dict()}), 200\n
        721 |             else:\n
        72
```

```

    },
    "end": {
        "line": 70,
        "col": 91,
        "offset": 2160
    },
    "extra": {
        "message": "Detected user input used to manually construct a SQL string. This is usually bad practice",
        "metadata": {
            "cwe": [
                "CWE-704: Incorrect Type Conversion or Cast"
            ],
            "owasp": [
                "A01:2017 - Injection",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "references": [
                "https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-textual-sql",
                "https://www.tutorialspoint.com/sqlalchemy/sqlalchemy_quick_guide.htm",
                "https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-more-specific-text-with-table-expressions"
            ],
            "category": "security",
            "technology": [
                "sqlalchemy",
                "flask"
            ],
            "subcategory": [
                "vuln"
            ],
            "impact": "MEDIUM",
            "likelihood": "MEDIUM",
            "confidence": "MEDIUM",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [
                "Improper Validation"
            ],
            "source": "https://semgrep.dev/r/python.flask.security.injection.tainted-sql-string.tainted-sql-string",
            "shortlink": "https://sg.run/JxZj"
        },
        "severity": "ERROR",
        "fingerprint": "requires login",
        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "ffeb1332-27c7-4f01-bda3-c2acc4735f49",
    "title": "Sql Injection in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 68,
    "line_end": 81,
    "severity": "MEDIUM",
    "vuln_type": "SQL_INJECTION",
    "description": "User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection.",
    "rule_id": "python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db-cursor-execute",
    "confidence": 0.7,
    "cwe_id": "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')",
    "cvss_score": null,
    "code_snippet": "
        66 |         error = None\n
        67 |         if request.method == \"POST\":\n
        68 |             data = request.get_json()\n
        69 |             if data is not None:\n
        70 |                 username = data.get('username')\n
        71 |                 password = data.get('password')\n
        72 |                 if not username or not password:\n
        73 |                     error = 'Invalid input'\n
        74 |                     return jsonify({'error': error})\n
        75 |                 cursor.execute('SELECT * FROM users WHERE username = %s AND password = %s', (username, password))\n
        76 |                 user = cursor.fetchone()\n
        77 |                 if user:\n
        78 |                     session.login(user)\n
        79 |                     return jsonify({'message': 'Login successful'})\n
        80 |                 else:\n
        81 |                     error = 'Invalid credentials'\n
        82 |                     return jsonify({'error': error})\n
        83 |     ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.517,
    "risk_category": "HIGH",
    "category_source": "vuln_floor",
    "category_reason": "minimum floor for SQL_INJECTION",
    "llm_explanation": "SQL Injection occurs when user-controlled input is directly concatenated into SQL queries. In Python, use placeholders for user input instead of string concatenation.",
    "fix_suggestion": "Use parameterized queries instead of string concatenation. In Python, use placeholders for user input instead of string concatenation."
}

```

```

"business_impact": "An attacker could read, modify, or delete all data in the database, potentially exposing sensitive information.",
"raw": {
  "check_id": "python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection",
  "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "start": {
    "line": 68,
    "col": 9,
    "offset": 1978
  },
  "end": {
    "line": 81,
    "col": 51,
    "offset": 2624
  },
  "extra": {
    "message": "User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection.",
    "metadata": {
      "cwe": [
        "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')"
      ],
      "owasp": [
        "A01:2017 - Injection",
        "A03:2021 - Injection",
        "A05:2025 - Injection"
      ],
      "references": [
        "https://docs.djangoproject.com/en/3.0/topics/security/#sql-injection-protection"
      ],
      "category": "security",
      "technology": [
        "django"
      ],
      "cwe2022-top25": true,
      "cwe2021-top25": true,
      "subcategory": [
        "vuln"
      ],
      "likelihood": "MEDIUM",
      "impact": "HIGH",
      "confidence": "MEDIUM",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "SQL Injection"
      ],
      "source": "https://semgrep.dev/r/python.django.security.injection.sql.sql-injection-using-db-cursor-execute",
      "shortlink": "https://sg.run/qx7y"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
},
{
  "id": "7e898ca5-de81-4502-9529-e2ffb34ff758",
  "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\admin.html",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\admin.html",
  "line_start": 22,
  "line_end": 22,
  "severity": "MEDIUM",
  "vuln_type": "XSS",
  "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False'. This could lead to a Cross-site Scripting (XSS) attack.",
  "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
  "confidence": 0.7,
  "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
  "cvss_score": null,
  "code_snippet": "
    20 |         <button class='btn primary' type='submit'>Run</button>
    21 |     "
  "sources": [
    "semgrep"
  ]
}

```

```

],
"risk_score": 0.4747,
"risk_category": "MEDIUM",
"category_source": "score",
"category_reason": "score-threshold",
"llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output",
"fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in HTML",
"business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or perform other unauthorized actions",
"raw": {
  "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
  "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\admin.html",
  "start": {
    "line": 22,
    "col": 26,
    "offset": 923
  },
  "end": {
    "line": 22,
    "col": 51,
    "offset": 948
  },
  "extra": {
    "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with `render`",
    "metadata": {
      "cwe": [
        "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')"
      ],
      "owasp": [
        "A07:2017 - Cross-Site Scripting (XSS)",
        "A03:2021 - Injection",
        "A05:2025 - Injection"
      ],
      "references": [
        "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
      ],
      "category": "security",
      "technology": [
        "flask"
      ],
      "cwe2022-top25": true,
      "cwe2021-top25": true,
      "subcategory": [
        "audit"
      ],
      "likelihood": "LOW",
      "impact": "MEDIUM",
      "confidence": "LOW",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "Cross-Site-Scripting (XSS)"
      ],
      "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
      "shortlink": "https://sg.run/W8og"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
}
},
{
  "id": "728f4133-56d3-4ce8-99a3-eb89b8b16342",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:10",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 10,
  "line_end": 10,
  "severity": "HIGH",
  "vuln_type": "UNKNOWN",
  "description": "The Python documentation recommends using `defusedxml` instead of `xml` because the native `xml` module is vulnerable to a Denial of Service (DoS) attack. The attack is triggered by parsing a large, untrusted XML document, which can cause the application to run out of memory or become unresponsive. The fix is to use the `defusedxml` module, which is a fork of the standard `xml` module that is designed to be resistant to DoS attacks."
}

```

```

"rule_id": "python.lang.security.use-defused-xml.use-defused-xml",
"confidence": 0.9,
"cwe_id": "CWE-611: Improper Restriction of XML External Entity Reference",
"cvss_score": null,
"code_snippet": "
            8 | import subprocess\n
            9 | import urllib.request\n>>> 10 | import xml.e
"sources": [
    "semgrep"
],
"risk_score": 0.4508,
"risk_category": "MEDIUM",
"category_source": "score",
"category_reason": "score-threshold",
"llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be
"fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelin
"business_impact": "This vulnerability could be exploited to compromise application security and potent
"raw": {
    "check_id": "python.lang.security.use-defused-xml.use-defused-xml",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "start": {
        "line": 10,
        "col": 1,
        "offset": 148
    },
    "end": {
        "line": 10,
        "col": 35,
        "offset": 182
    },
    "extra": {
        "message": "The Python documentation recommends using `defusedxml` instead of `xml` because the nat
        "metadata": {
            "owasp": [
                "A04:2017 - XML External Entities (XXE)",
                "A05:2021 - Security Misconfiguration",
                "A02:2025 - Security Misconfiguration"
            ],
            "cwe": [
                "CWE-611: Improper Restriction of XML External Entity Reference"
            ],
            "references": [
                "https://docs.python.org/3/library/xml.html",
                "https://github.com/tiran/defusedxml",
                "https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processing"
            ],
            "category": "security",
            "technology": [
                "python"
            ],
            "cwe2022-top25": true,
            "cwe2021-top25": true,
            "subcategory": [
                "audit"
            ],
            "likelihood": "LOW",
            "impact": "MEDIUM",
            "confidence": "LOW",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [
                "XML Injection"
            ],
            "source": "https://semgrep.dev/r/python.lang.security.use-defused-xml.use-defused-xml",
            "shortlink": "https://sg.run/kX47"
        },
        "severity": "ERROR",
        "fingerprint": "requires login",
        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},

```

```

{
  "id": "0a94a8b9-99c3-405c-b009-ca979a6067e3",
  "title": "Open Redirect in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 87,
  "line_end": 88,
  "severity": "HIGH",
  "vuln_type": "OPEN_REDIRECT",
  "description": "Data from request is passed to redirect(). This is an open redirect and could be exploited",
  "rule_id": "python.flask.security.open-redirect.open-redirect",
  "confidence": 0.9,
  "cwe_id": "CWE-601: URL Redirection to Untrusted Site ('Open Redirect')",
  "cvss_score": null,
  "code_snippet": "
      85 | @app.route(\"/logout\")\n
      86 | def logout():\n>>> 87 |     next_url =
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.4508,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "Open redirect vulnerabilities allow attackers to redirect users to malicious websites",
  "fix_suggestion": "Validate redirect URLs against a whitelist of allowed domains. Use relative paths instead of absolute paths",
  "business_impact": "Attackers could use your application to redirect users to phishing sites, potentially stealing sensitive information",
  "raw": {
    "check_id": "python.flask.security.open-redirect.open-redirect",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "start": {
      "line": 87,
      "col": 5,
      "offset": 2756
    },
    "end": {
      "line": 88,
      "col": 34,
      "offset": 2844
    },
    "extra": {
      "message": "Data from request is passed to redirect(). This is an open redirect and could be exploited",
      "metadata": {
        "cwe": [
          "CWE-601: URL Redirection to Untrusted Site ('Open Redirect')"
        ],
        "owasp": [
          "A01:2021 - Broken Access Control",
          "A01:2025 - Broken Access Control"
        ],
        "references": [
          "https://flask-login.readthedocs.io/en/latest/#login-example",
          "https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html",
          "https://docs.python.org/3/library/urllib.parse.html#url-parsing"
        ],
        "category": "security",
        "technology": [
          "flask"
        ],
        "subcategory": [
          "audit"
        ],
        "likelihood": "LOW",
        "impact": "MEDIUM",
        "confidence": "LOW",
        "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
        "vulnerability_class": [
          "Open Redirect"
        ],
        "source": "https://semgrep.dev/r/python.flask.security.open-redirect.open-redirect",
        "shortlink": "https://sg.run/kXe2"
      },
      "severity": "ERROR",
      "fingerprint": "requires login",

```

```

        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "97f01841-af17-4d8e-93cd-436c1085da3b",
    "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:313",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 313,
    "line_end": 313,
    "severity": "HIGH",
    "vuln_type": "UNKNOWN",
    "description": "Detected user input entering a `subprocess` call unsafely. This could result in a command execution.",
    "rule_id": "python.flask.security.injection.subprocess-injection.subprocess-injection",
    "confidence": 0.9,
    "cwe_id": "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')",
    "cvss_score": null,
    "code_snippet": "
        311 |         cmd = request.args.get(\"cmd\", \"status\")\n
        312 |         shell_cmd =
    ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4508,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be exploited to compromise application security and potentially cause data loss or system downtime.",
    "fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelines for more information on preventing command injection attacks.",
    "business_impact": "This vulnerability could be exploited to compromise application security and potentially cause data loss or system downtime.",
    "raw": {
        "check_id": "python.flask.security.injection.subprocess-injection.subprocess-injection",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
        "start": {
            "line": 313,
            "col": 12,
            "offset": 11098
        },
        "end": {
            "line": 313,
            "col": 81,
            "offset": 11167
        }
    },
    "extra": {
        "message": "Detected user input entering a `subprocess` call unsafely. This could result in a command execution.",
        "metadata": {
            "category": "security",
            "technology": [
                "flask"
            ],
            "owasp": [
                "A01:2017 - Injection",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "cwe": [
                "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')"
            ],
            "references": [
                "https://semgrep.dev/docs/cheat-sheets/python-command-injection/"
            ],
            "confidence": "HIGH",
            "cwe2022-top25": true,
            "cwe2021-top25": true,
            "subcategory": [
                "vuln"
            ],
            "likelihood": "HIGH",
            "impact": "MEDIUM",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [

```

```

        "Command Injection"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.injection.subprocess-injection.subprocess-",
    "shortlink": "https://sg.run/5gw3"
},
"severity": "ERROR",
"fingerprint": "requires login",
"lines": "requires login",
"validation_state": "NO_VALIDATOR",
"engine_kind": "OSS"
}
}
},
{
    "id": "80bfef2e-901a-4242-919d-bf6f53c7e346",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\account.html",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\account.html",
    "line_start": 24,
    "line_end": 24,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False'.",
    "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "
        22 |         {% for tx in transactions %}
        23 |             <tr>
        24 |                 <td>{{ tx.name }}</td>
    ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4042,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper escaping.",
    "fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in HTML.",
    "business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or perform other unauthorized actions.",
    "raw": {
        "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\account.html",
        "start": {
            "line": 24,
            "col": 13,
            "offset": 667
        },
        "end": {
            "line": 24,
            "col": 39,
            "offset": 693
        }
    },
    "extra": {
        "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False'.",
        "metadata": {
            "cwe": [
                "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')"
            ],
            "owasp": [
                "A07:2017 - Cross-Site Scripting (XSS)",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "references": [
                "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
            ],
            "category": "security",
            "technology": [
                "flask"
            ],
            "cwe2022-top25": true,
            "cwe2021-top25": true,
            "subcategory": [

```

```

        "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
        "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.tem",
    "shortlink": "https://sg.run/W8og"
},
{
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
}
}
},
{
    "id": "a5ca7032-3b6c-44d7-81b6-1e20b5f62801",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\dashb",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\dashboard.htm",
    "line_start": 61,
    "line_end": 61,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with '
    "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "
        59 |
        <td>{{ tx.counterparty }}</td>\n
        60 |
        <td>\u20b9{{
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4042,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output",
    "fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in H",
    "business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or p",
    "raw": {
        "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\dashboard.h",
        "start": {
            "line": 61,
            "col": 15,
            "offset": 2039
        },
        "end": {
            "line": 61,
            "col": 33,
            "offset": 2057
        },
        "extra": {
            "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with '
            "metadata": {
                "cwe": [
                    "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
                ],
                "owasp": [
                    "A07:2017 - Cross-Site Scripting (XSS)",
                    "A03:2021 - Injection",
                    "A05:2025 - Injection"
                ],
                "references": [
                    "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
                ],
            },
        },
    },

```

```

"category": "security",
"technology": [
  "flask"
],
"cwe2022-top25": true,
"cwe2021-top25": true,
"subcategory": [
  "audit"
],
"likelihood": "LOW",
"impact": "MEDIUM",
"confidence": "LOW",
"license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
"vulnerability_class": [
  "Cross-Site-Scripting (XSS)"
],
"source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
"shortlink": "https://sg.run/W8og"
},
"severity": "WARNING",
"fingerprint": "requires login",
"lines": "requires login",
"validation_state": "NO_VALIDATOR",
"engine_kind": "OSS"
}
},
{
  "id": "4bc7ba8a-9e42-431f-934f-e8e38a0c145f",
  "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\reflect.html",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\reflect.html",
  "line_start": 7,
  "line_end": 7,
  "severity": "MEDIUM",
  "vuln_type": "XSS",
  "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False' in the render function call.",
  "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
  "confidence": 0.7,
  "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
  "cvss_score": null,
  "code_snippet": "5 | <h1>Customer Notice Board</h1>\n6 | <p class=\"subtle\">This page is under construction.\n7 | </p>\n8 | </div>\n9 | </div>\n10 | </div>\n11 | </div>\n12 | </div>\n13 | </div>\n14 | </div>\n15 | </div>\n16 | </div>\n17 | </div>\n18 | </div>\n19 | </div>\n20 | </div>\n21 | </div>\n22 | </div>\n23 | </div>\n24 | </div>\n25 | </div>\n26 | </div>\n27 | </div>\n28 | </div>\n29 | </div>\n30 | </div>\n31 | </div>\n32 | </div>\n33 | </div>\n34 | </div>\n35 | </div>\n36 | </div>\n37 | </div>\n38 | </div>\n39 | </div>\n40 | </div>\n41 | </div>\n42 | </div>\n43 | </div>\n44 | </div>\n45 | </div>\n46 | </div>\n47 | </div>\n48 | </div>\n49 | </div>\n50 | </div>\n51 | </div>\n52 | </div>\n53 | </div>\n54 | </div>\n55 | </div>\n56 | </div>\n57 | </div>\n58 | </div>\n59 | </div>\n60 | </div>\n61 | </div>\n62 | </div>\n63 | </div>\n64 | </div>\n65 | </div>\n66 | </div>\n67 | </div>\n68 | </div>\n69 | </div>\n70 | </div>\n71 | </div>\n72 | </div>\n73 | </div>\n74 | </div>\n75 | </div>\n76 | </div>\n77 | </div>\n78 | </div>\n79 | </div>\n80 | </div>\n81 | </div>\n82 | </div>\n83 | </div>\n84 | </div>\n85 | </div>\n86 | </div>\n87 | </div>\n88 | </div>\n89 | </div>\n90 | </div>\n91 | </div>\n92 | </div>\n93 | </div>\n94 | </div>\n95 | </div>\n96 | </div>\n97 | </div>\n98 | </div>\n99 | </div>\n100 | </div>\n101 | </div>\n102 | </div>\n103 | </div>\n104 | </div>\n105 | </div>\n106 | </div>\n107 | </div>\n108 | </div>\n109 | </div>\n110 | </div>\n111 | </div>\n112 | </div>\n113 | </div>\n114 | </div>\n115 | </div>\n116 | </div>\n117 | </div>\n118 | </div>\n119 | </div>\n120 | </div>\n121 | </div>\n122 | </div>\n123 | </div>\n124 | </div>\n125 | </div>\n126 | </div>\n127 | </div>\n128 | </div>\n129 | </div>\n130 | </div>\n131 | </div>\n132 | </div>\n133 | </div>\n134 | </div>\n135 | </div>\n136 | </div>\n137 | </div>\n138 | </div>\n139 | </div>\n140 | </div>\n141 | </div>\n142 | </div>\n143 | </div>\n144 | </div>\n145 | </div>\n146 | </div>\n147 | </div>\n148 | </div>\n149 | </div>\n150 | </div>\n151 | </div>\n152 | </div>\n153 | </div>\n154 | </div>\n155 | </div>\n156 | </div>\n157 | </div>\n158 | </div>\n159 | </div>\n160 | </div>\n161 | </div>\n162 | </div>\n163 | </div>\n164 | </div>\n165 | </div>\n166 | </div>\n167 | </div>\n168 | </div>\n169 | </div>\n170 | </div>\n171 | </div>\n172 | </div>\n173 | </div>\n174 | </div>\n175 | </div>\n176 | </div>\n177 | </div>\n178 | </div>\n179 | </div>\n180 | </div>\n181 | </div>\n182 | </div>\n183 | </div>\n184 | </div>\n185 | </div>\n186 | </div>\n187 | </div>\n188 | </div>\n189 | </div>\n190 | </div>\n191 | </div>\n192 | </div>\n193 | </div>\n194 | </div>\n195 | </div>\n196 | </div>\n197 | </div>\n198 | </div>\n199 | </div>\n200 | </div>\n201 | </div>\n202 | </div>\n203 | </div>\n204 | </div>\n205 | </div>\n206 | </div>\n207 | </div>\n208 | </div>\n209 | </div>\n210 | </div>\n211 | </div>\n212 | </div>\n213 | </div>\n214 | </div>\n215 | </div>\n216 | </div>\n217 | </div>\n218 | </div>\n219 | </div>\n220 | </div>\n221 | </div>\n222 | </div>\n223 | </div>\n224 | </div>\n225 | </div>\n226 | </div>\n227 | </div>\n228 | </div>\n229 | </div>\n230 | </div>\n231 | </div>\n232 | </div>\n233 | </div>\n234 | </div>\n235 | </div>\n236 | </div>\n237 | </div>\n238 | </div>\n239 | </div>\n240 | </div>\n241 | </div>\n242 | </div>\n243 | </div>\n244 | </div>\n245 | </div>\n246 | </div>\n247 | </div>\n248 | </div>\n249 | </div>\n250 | </div>\n251 | </div>\n252 | </div>\n253 | </div>\n254 | </div>\n255 | </div>\n256 | </div>\n257 | </div>\n258 | </div>\n259 | </div>\n260 | </div>\n261 | </div>\n262 | </div>\n263 | </div>\n264 | </div>\n265 | </div>\n266 | </div>\n267 | </div>\n268 | </div>\n269 | </div>\n270 | </div>\n271 | </div>\n272 | </div>\n273 | </div>\n274 | </div>\n275 | </div>\n276 | </div>\n277 | </div>\n278 | </div>\n279 | </div>\n280 | </div>\n281 | </div>\n282 | </div>\n283 | </div>\n284 | </div>\n285 | </div>\n286 | </div>\n287 | </div>\n288 | </div>\n289 | </div>\n290 | </div>\n291 | </div>\n292 | </div>\n293 | </div>\n294 | </div>\n295 | </div>\n296 | </div>\n297 | </div>\n298 | </div>\n299 | </div>\n300 | </div>\n301 | </div>\n302 | </div>\n303 | </div>\n304 | </div>\n305 | </div>\n306 | </div>\n307 | </div>\n308 | </div>\n309 | </div>\n310 | </div>\n311 | </div>\n312 | </div>\n313 | </div>\n314 | </div>\n315 | </div>\n316 | </div>\n317 | </div>\n318 | </div>\n319 | </div>\n320 | </div>\n321 | </div>\n322 | </div>\n323 | </div>\n324 | </div>\n325 | </div>\n326 | </div>\n327 | </div>\n328 | </div>\n329 | </div>\n330 | </div>\n331 | </div>\n332 | </div>\n333 | </div>\n334 | </div>\n335 | </div>\n336 | </div>\n337 | </div>\n338 | </div>\n339 | &lt
```

```

        "A07:2017 - Cross-Site Scripting (XSS)",
        "A03:2021 - Injection",
        "A05:2025 - Injection"
    ],
    "references": [
        "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
    ],
    "category": "security",
    "technology": [
        "flask"
    ],
    "cwe2022-top25": true,
    "cwe2021-top25": true,
    "subcategory": [
        "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
        "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "shortlink": "https://sg.run/W8og"
},
{
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
}
},
{
    "id": "cb514dd8-c52e-4b23-bfba-b245b43f1607",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\support.html",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\support.html",
    "line_start": 23,
    "line_end": 23,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected a segment of a Flask template where autoescaping is explicitly disabled with 'autoescape=False'. This is a security vulnerability as it allows for Cross-Site Scripting (XSS) attacks.",
    "rule_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "
21 |         <td>{{ ticket.name }}</td>\n
22 |         <td>{{ ticket.sub
",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.4042,
    "risk_category": "MEDIUM",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper escaping, allowing attackers to inject malicious scripts.",
    "fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in HTML.",
    "business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or perform other actions depending on the context of the injected script.",
    "raw": {
        "check_id": "python.flask.security.xss.audit.template-unescaped-with-safe.template-unescaped-with-safe",
        "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\templates\\support.html",
        "start": {
            "line": 23,
            "col": 15,
            "offset": 853
        },
        "end": {
            "line": 23,
            "col": 37,
            "offset": 875
        }
    },
},

```

```

"extra": {
  "message": "Detected a segment of a Flask template where autoescaping is explicitly disabled with '
  "metadata": {
    "cwe": [
      "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    ],
    "owasp": [
      "A07:2017 - Cross-Site Scripting (XSS)",
      "A03:2021 - Injection",
      "A05:2025 - Injection"
    ],
    "references": [
      "https://flask.palletsprojects.com/en/1.1.x/security/#cross-site-scripting-xss"
    ],
    "category": "security",
    "technology": [
      "flask"
    ],
    "cwe2022-top25": true,
    "cwe2021-top25": true,
    "subcategory": [
      "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
      "Cross-Site-Scripting (XSS)"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.xss.audit.template-unescaped-with-safe.tem
    "shortlink": "https://sg.run/W8og"
  },
  "severity": "WARNING",
  "fingerprint": "requires login",
  "lines": "requires login",
  "validation_state": "NO_VALIDATOR",
  "engine_kind": "OSS"
}
}
},
{
  "id": "90f85928-8833-441d-9c38-601f135fe935",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:248",
  "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
  "line_start": 248,
  "line_end": 248,
  "severity": "MEDIUM",
  "vuln_type": "UNKNOWN",
  "description": "Detected a dynamic value being used with urllib. urllib supports 'file://' schemes, so a
  "rule_id": "python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected",
  "confidence": 0.7,
  "cwe_id": "CWE-939: Improper Authorization in Handler for Custom URL Scheme",
  "cvss_score": null,
  "code_snippet": "      246 | def fetch_remote():\n      247 |         target = request.args.get(\"url\", \"ht
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.3384,
  "risk_category": "LOW",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be
  "fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelin
  "business_impact": "This vulnerability could be exploited to compromise application security and potent
  "raw": {
    "check_id": "python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected",
    "path": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "start": {
      "line": 248,
      "col": 10,

```

```

        "offset": 8376
    },
    "end": {
        "line": 248,
        "col": 51,
        "offset": 8417
    },
    "extra": {
        "message": "Detected a dynamic value being used with urllib. urllib supports 'file://\' schemes, so a",
        "metadata": {
            "cwe": [
                "CWE-939: Improper Authorization in Handler for Custom URL Scheme"
            ],
            "owasp": "A01:2017 - Injection",
            "source-rule-url": "https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee954c2b1",
            "bandit-code": "B310",
            "asvs": {
                "control_id": "5.2.4 Dynamic Code Execution Features",
                "control_url": "https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-Sanitization",
                "section": "V5: Validation, Sanitization and Encoding Verification Requirements",
                "version": "4"
            },
            "category": "security",
            "technology": [
                "python"
            ],
            "references": [
                "https://cwe.mitre.org/data/definitions/939.html"
            ],
            "subcategory": [
                "audit"
            ],
            "likelihood": "LOW",
            "impact": "LOW",
            "confidence": "LOW",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [
                "Improper Authorization"
            ],
            "source": "https://semgrep.dev/r/python.lang.security.audit.dynamic-urllib-use-detected.dynamic-u",
            "shortlink": "https://sg.run/dKZZ"
        },
        "severity": "WARNING",
        "fingerprint": "requires login",
        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "8d9c9878-4166-4d1f-94d9-626faadc977e",
    "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py:369",
    "file": "C:\\Users\\Dhruv\\Desktop\\CyberScurity Project\\Test\\vuln_bank_lab\\app.py",
    "line_start": 369,
    "line_end": 369,
    "severity": "MEDIUM",
    "vuln_type": "UNKNOWN",
    "description": "Detected Flask app with debug=True. Do not deploy to production with this flag enabled a",
    "rule_id": "python.flask.security.audit.debug-enabled.debug-enabled",
    "confidence": 0.7,
    "cwe_id": "CWE-489: Active Debug Code",
    "cvss_score": null,
    "code_snippet": "
        367 | \n
        368 | if __name__ == \"__main__\":\n>>>
        369 |     app.run(host=\"127.0.0.1\", port=5000)
    ",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.282,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",

```

```

"llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be
"fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelin
"business_impact": "This vulnerability could be exploited to compromise application security and potent
"raw": {
  "check_id": "python.flask.security.audit.debug-enabled.debug-enabled",
  "path": "C:\\Users\\Dhruv\\Desktop\\CyberSecurity Project\\Test\\vuln_bank_lab\\app.py",
  "start": {
    "line": 369,
    "col": 5,
    "offset": 12789
  },
  "end": {
    "line": 369,
    "col": 53,
    "offset": 12837
  },
  "extra": {
    "message": "Detected Flask app with debug=True. Do not deploy to production with this flag enabled a
    "metadata": {
      "cwe": [
        "CWE-489: Active Debug Code"
      ],
      "owasp": "A06:2017 - Security Misconfiguration",
      "references": [
        "https://labs.detectify.com/2015/10/02/how-patreon-got-hacked-publicly-exposed-werkzeug-debugger
      ],
      "category": "security",
      "technology": [
        "flask"
      ],
      "subcategory": [
        "vuln"
      ],
      "likelihood": "HIGH",
      "impact": "MEDIUM",
      "confidence": "HIGH",
      "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
      "vulnerability_class": [
        "Active Debug Code"
      ],
      "source": "https://semgrep.dev/r/python.flask.security.audit.debug-enabled.debug-enabled",
      "shortlink": "https://sg.run/dKrd"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
  }
}
}
]

```